A.I. Image Detector

By Team 2: Yasin Ahmed, Kaiye Liu, & Tyler Rimkunas

Problem Statement 3
Data Set Description 3
Data preparation 6
Model and/or technique selection 7
Evaluation 9
Error analysis 11
Effort to improve results 14
Conclusions and future work 14

Problem Statement

With the exponential rise in popularity of using A.I. image generation, social media is overwhelmed with A.I. generated content. Throughout all of social media, over 70% of images posted are now A.I. generated, and 78% of users are finding it harder to distinguish between A.I. and human. Given its efficiency in tricking the average viewer, this tool can be weaponized for misinformation and fabrication which can cause widespread panic or even violence. The goal of this project is to build a machine learning model that can accurately differentiate human generated images and A.I. generated images. This can be classified as a binary classification problem with one class being human generated and the other being A.I. generated. The target is to successfully and accurately classify at least 80% of the 11,992 images with half being human generated and the other half being A.I. generated.

Data Set Description

The dataset used is the AI vs. Human-Generated Images dataset, published on Kaggle by Alessandra Sala, Manuela Jeyaraj, Toma Ijatomi, & Margarita Pitsiani. The dataset was specifically constructed to support binary image classification research. The human-generated images were sourced from Shutterstock and span a wide variety of categories, with approximately one-third featuring human subjects. Each authentic image is paired with a counterpart produced by a generative AI model, enabling direct comparison between real and synthetic visual content.

The selected data set contains images that were placed in two folders: “testing data” and “training data”. There are also two comma separated values (csv) files for both training data (Figure 1) and testing data (Figure 2) that contain the file path (train/file_name.jpg). However, the training data csv contains label columns to indicate if the image is either A.I. generated(1) or human generated(0) but the testing data csv does not. In total, there are 79,950 images in the training data and 5,540 images in the testing data. Half of the training data images are human generated and half are A.I. generated. It is not clarified the ratio from human generated images to A.I. generated images in testing data.

	Unnamed: 0	file_name	label
0	0	train_data/a6dcb93f596a43249135678dfcfc17ea.jpg	1
1	1	train_data/041be3153810433ab146bc97d5af505c.jpg	0
2	2	train_data/615df26ce9494e5db2f70e57ce7a3a4f.jpg	1
3	3	train_data/8542fe161d9147be8e835e50c0de39cd.jpg	0
4	4	train_data/5d81fa12bc3b4cea8c94a6700a477cf2.jpg	1

Figure 1: The screenshot above shows the top of train.csv

	id
0	test_data_v2/1a2d9fd3e21b4266aea1f66b30aed157.jpg
1	test_data_v2/ab5df8f441fe4fbf9dc9c6baae699dc7.jpg
2	test_data_v2/eb364dd2dfe34feda0e52466b7ce7956.jpg
3	test_data_v2/f76c2580e9644d85a741a42c6f6b39c0.jpg
4	test_data_v2/a16495c578b7494683805484ca27cf9f.jpg

Figure 2: The screenshot above shows the top of test.csv



Figure 3: The image above is an example of a human generated image from the training data.



Figure 4: The image above is an example of an A.I. generated image from the training data.

Data preparation

To prepare the data, the “testing data” folder and its associated csv was dropped since the images were not labeled as human generated or A.I. generated. Instead, the “training data” folder was split by a python script into 3 directories: Train(70%), Validation(~15%), and Test(~15%).

Figure 5: The table below showcases the split of the data set into training, validation, and test sets:

Split	Images	Notes
Train	55,965	Used for model training
Validation	11,993	Used during training to monitor overfitting
Test	11,992	Held out; used only for final evaluation

Within each split, images are organized into two subdirectories: “human_images” and “ai_images”. With the exception of one-offs due to an odd amount of images, each subdirectory had an equal amount of images to their opposing subdirectory. This dataset differs from the earlier CSV-based version because the directory-based structure proved far better suited to TensorFlow’s image_dataset_from_directory utility, enabling efficient batch loading at full resolution without the manual per-image file I/O that caused memory and performance bottlenecks in the initial approach.

Given that the images varied in resolution, all of them were resized to 224x224, a standard input size well-suited for convolutional neural networks and consistent with widely-used pretrained architectures. Pixel values are rescaled from the original [0, 255] range to [0, 1] using a Rescaling(1./255) layer as the first layer of the model. This normalization is essential for stable gradient flow during training. A batch size of 32 is used for training and evaluation, balancing memory usage against training throughput on the A100 GPU used in Google Colab Pro. Since the classes are organized alphabetically automatically, they are explicitly set to ['human_images', 'ai_images'] so that label 0 corresponds to human and label 1 corresponds to AI, matching the labeling convention from the original dataset and mitigating confusion later on for the Gradient-weighted Class Activation Mappings.

Model and/or technique selection

The natural choice for a model was a convolutional neural network, as CNNs are specifically designed to exploit the spatial hierarchy of features in images. Rather than requiring manually engineered features, convolutional filters learn to detect edges, textures, patterns, and higher-level visual concepts automatically through end-to-end training.

Figure 6: The final model is a five-block custom CNN built in TensorFlow/Keras with the following architecture:

Layer	Output Shape	Params
Input + Rescaling	(None, 224, 224, 3)	0
Conv2D (32 filters, 3x3) + MaxPool	(None, 111, 111, 32)	896

Conv2D (64 filters, 3x3) + MaxPool	(None, 54, 54, 64)	18,496
Conv2D (128 filters, 3x3) + MaxPool	(None, 26, 26, 128)	73,856
Conv2D (256 filters, 3x3) + MaxPool	(None, 12, 12, 256)	295,168
Conv2D (256 filters, 3x3) + Flatten	(None, 10, 10, 256)	590,080
Dropout (0.5)	(None, 25,600)	0
Dense (1, sigmoid)	(None, 1)	25,601

Progressive filter doubling allows the network to capture increasingly abstract features at each stage. MaxPooling after each convolutional block reduces spatial dimensions, decreasing computational load and promoting translational invariance. Dropout(0.5) before the output layer regularizes the network to reduce overfitting on the large training set. A single Dense neuron with sigmoid activation produces a probability score in [0, 1]; predictions above 0.5 are classified as AI-generated. Binary cross-entropy loss is appropriate for this binary classification task. RMSprop optimizer with a conservative learning rate of 1e-5 was selected for stable convergence and to prevent overfitting. ModelCheckpoint saves the epoch with the lowest validation loss, ensuring the best generalization is retained. The model was trained for 30 epochs on the full training set of 55,965 images. The final, total parameters are 1,004,097 (3.83 MB).

As documented in the progress report, the initial approach applied a Stochastic Gradient Descent (SGD) classifier from scikit-learn, trained on 70,000 images that were converted to grayscale, resized to 28×28 pixels, and flattened into 784-dimensional feature vectors. This established a baseline but produced poor results: precision of 0.51 and recall of 0.64, essentially near-random performance. The core limitation was that flattening 28×28 grayscale pixels discards spatial structure entirely, leaving no meaningful features for the classifier to exploit.

Evaluation

During training of the model, the accuracy and loss of both the training set and the validation set are monitored to confirm that the accuracy is rising and the loss is lowering. If there is a plateau in the rise of validation accuracy, then it is a sign of overfitting. After the model is trained, the model evaluates itself on the test set and the accuracy is recorded. The target accuracy on the test set is at least 80%.

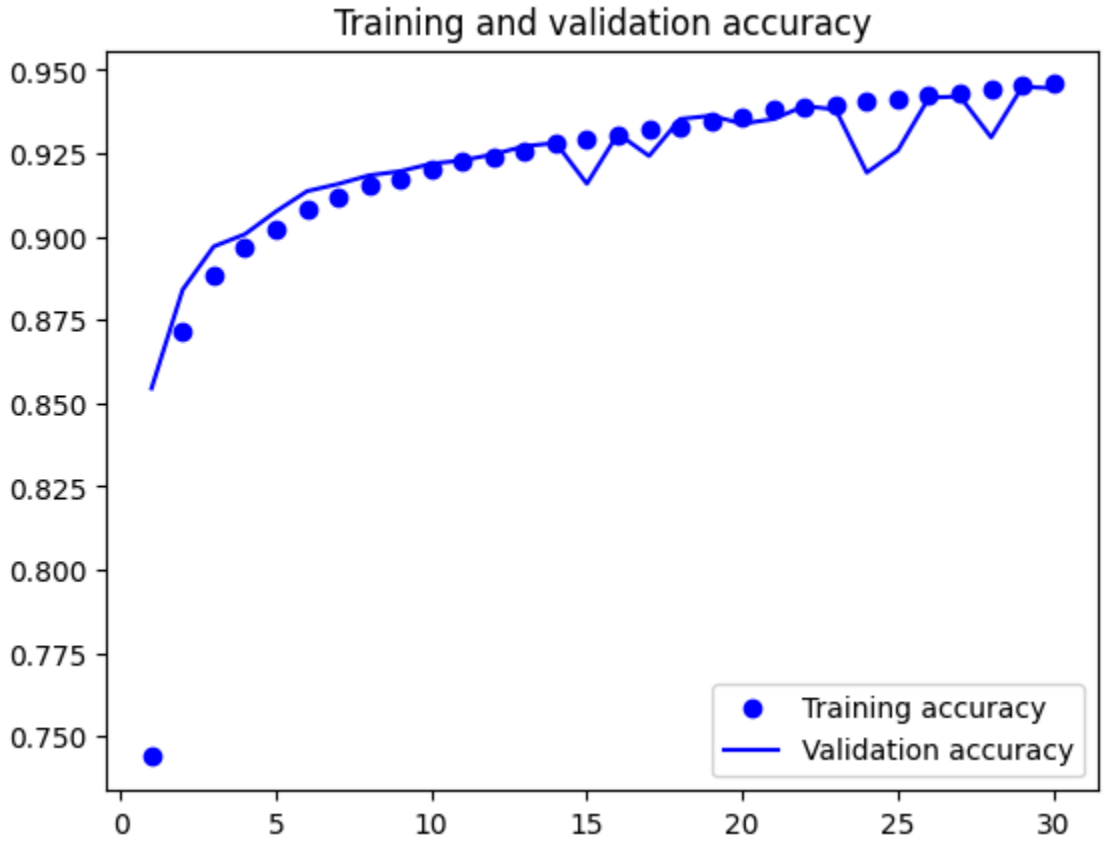


Figure 7: The image above is a plot of the accuracy of the training data and validation data for each epoch while training the model.

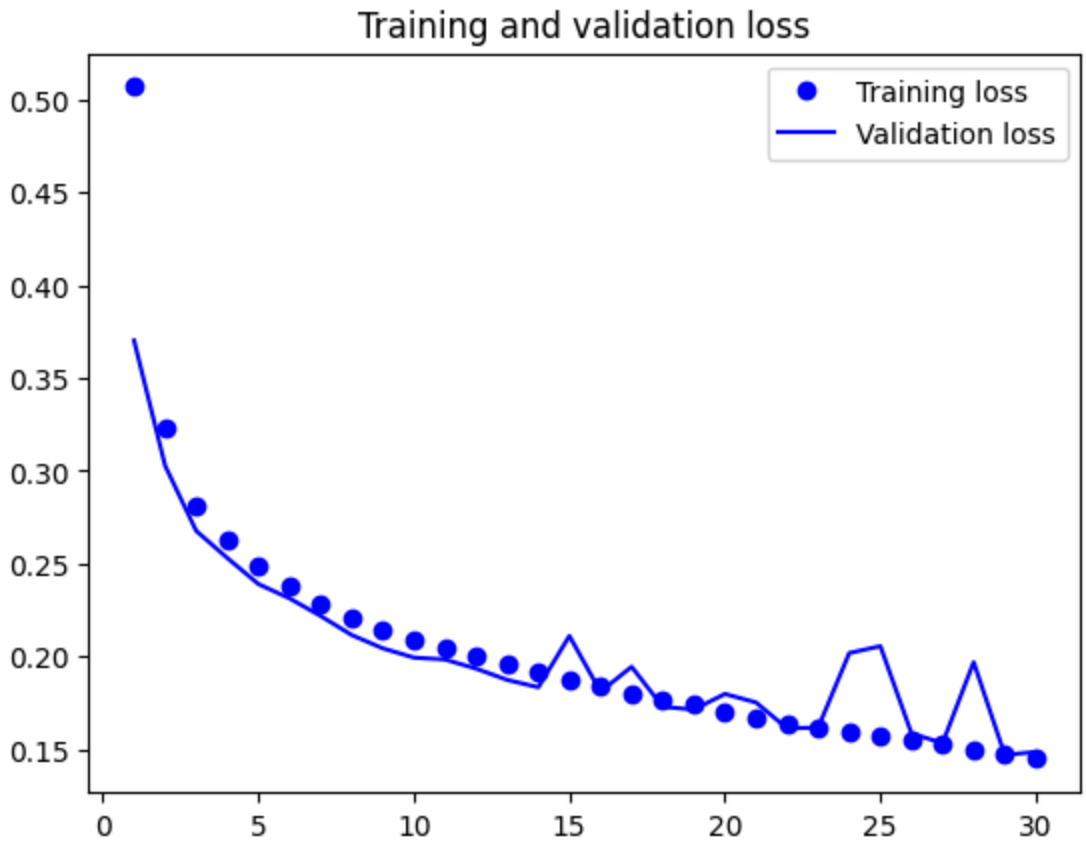


Figure 8: The image above is a plot of the loss of the training data and validation data for each epoch while training the model.

The model demonstrated consistent improvement throughout training. Training accuracy rose from 71.8% in epoch 1 to 94.6% by epoch 30. Validation accuracy followed closely, peaking at 94.86% at epoch 29 (the epoch saved by ModelCheckpoint, as it had the lowest validation loss of 0.1407). The gap between training and validation accuracy remained small throughout, indicating that dropout regularization successfully mitigated overfitting.
The recorded accuracy of the evaluation on the test set is 95.0%, achieving beyond the target accuracy.

Figure 9: After loading the best-checkpoint model (saved at epoch 29), the model was evaluated on the 11,992-image test set:

Class	Precision	Recall	F1-Score	Support
Human (0)	0.97	0.92	0.94	5,996

AI (1)	0.92	0.97	0.95	5,996
Macro Avg	0.95	0.94	0.94	11,992

As documented in the progress report, the initial approach applied a Stochastic Gradient Descent (SGD) classifier from Scikit-learn, trained on 70,000 images that were converted to grayscale, resized to 28×28 pixels, and flattened into 784-dimensional feature vectors. This established a baseline but produced a precision of 0.51 and recall of 0.64, essentially near-random performance. The core limitation was that flattening 28×28 grayscale pixels discards spatial structure entirely, leaving no meaningful features for the classifier to exploit. Additionally, the Scikit-learn package is not suited for utilizing a GPU for training and processing.

Figure 10: Comparison of the SGD model and the CNN:

Metric	Baseline (SGD)	Final CNN
Precision (AI)	0.51	0.92
Recall (AI)	0.64	0.97
Overall Accuracy	~50%	95.0%

The CNN far exceeded the progress report's target of 0.80 precision and recall, achieving 0.92 and 0.97 respectively for the AI class, representing a dramatic improvement over the baseline SGD approach.

Error analysis

The evaluation resulted in 486 false positives (Human images incorrectly classified as AI) and 178 false negatives (AI images incorrectly classified as human). The model is slightly more conservative about flagging AI images, which is a reasonable bias for a real-world detection tool where false positives may be more harmful than missed detections.

Figure 11: The confusion matrix on the 11,992-image test set (5,996 human + 5,996 AI):

	Predicted: Human	Predicted: A.I.
Actual: Human	TN = 5,510	FP = 486

Actual: AI	FN = 178	TP = 5,818

A sample of the false positives and false negatives were taken to process into a Gradient-weighted Class Activation Mapping, also known as a Grad-CAM, to study the CNN's thinking and reasoning for its estimation. The Grad-CAM analysis uses the last Conv2D layer (conv2d_4) to generate spatial heatmaps. Initial experiments with human test images showed very low AI-probability scores (e.g., 0.0018 for a correctly classified human image), with heatmaps highlighting edge regions and fine-textured areas as the key discriminating features. This suggests the model has learned to detect subtle pixel-level artifacts in A.I. generated images rather than relying on high-level semantic content, which is the expected and desirable behavior for this task.

In the sample of images that are classified as A.I. but are actually human, most are heavily edited drawings or computerized graphic design. Also, the absence of shadows in these images could heavily affect the results of the estimations.



Figure 12: The set of images above are the test images that were predicted by the model(top) as false positives with their corresponding probability of being A.I. and Grad-CAM(bottom).

The following image is an A.I. generated image that was classified as human generated. The model focused mostly on the face of the woman in the image than any other part of the image. During the resizing, the fingers in the image were also cut out, which possibly could have changed the outcome of this prediction. Also, given that this image also has shadows under the chin and near the ears, the model can be tricked into believing this is a human generated image.

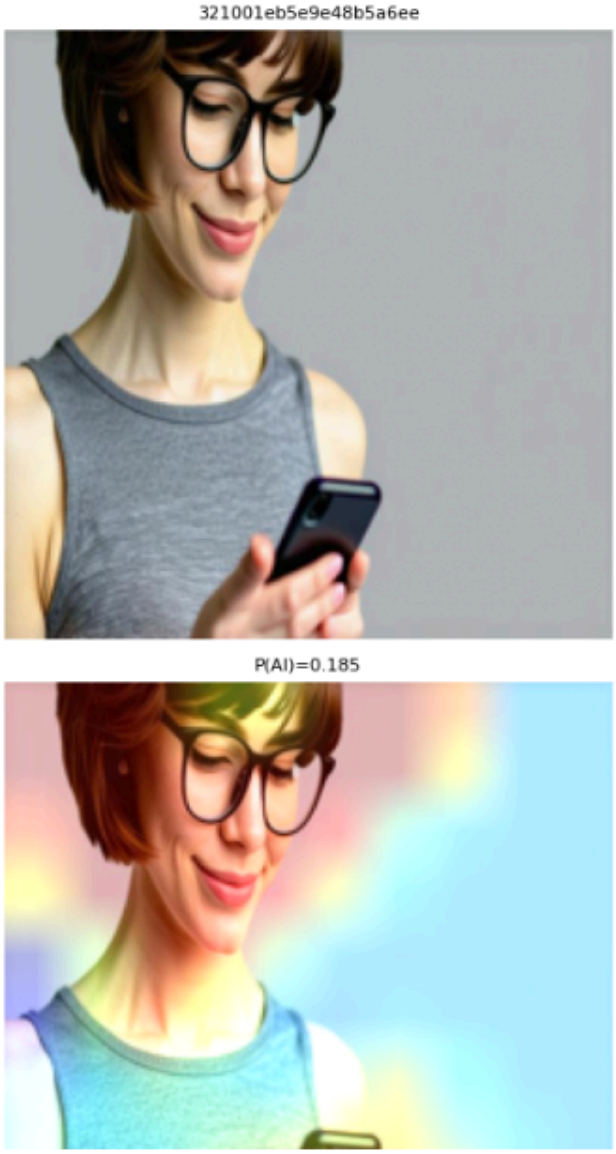


Figure 13: The images above are an example from the test data that was predicted by the model(top) as a false negative with its probability of being A.I. and corresponding Grad-CAM(bottom).

Effort to improve results

The progress report used a CSV-based dataset requiring manual PIL-based image loading at 28×28 grayscale, causing memory crashes and severe feature degradation. The final project adopted a directory-structured dataset compatible with TensorFlow's native data pipeline, enabling full-resolution 224×224 RGB loading with automatic prefetching, batching, and GPU utilization.

The flat, hand-crafted feature vector fed into an SGD classifier was replaced with a deep CNN that automatically learns hierarchical spatial features. The five convolutional blocks with progressive filter depth (32 to 256) allow the model to detect increasingly complex patterns.

A Dropout(0.5) layer was added before the output layer to prevent overfitting. The small and consistent gap between training and validation accuracy throughout the 30 epochs (never exceeding ~2%) confirms its effectiveness.

The project leveraged Google Colab Pro with an A100 GPU, which enabled training on 55,965 images at 224×224 resolution within approximately 28 seconds per epoch. This infrastructure upgrade was critical: the original approach crashed with out-of-memory errors and could not utilize GPU power.

Grad-CAM was added as a post-hoc interpretability tool, providing insight into model behavior beyond accuracy metrics. This is particularly important for a content-detection application where understanding model reasoning builds user trust.

Conclusions and future work

This project successfully developed a Convolutional Neural Network capable of classifying images as AI-generated or human-created with 95.0% overall test accuracy. Starting from a near-random SGD baseline (50% effective accuracy), the team progressively addressed each limitation: image resolution, feature extraction, training infrastructure, and model architecture.

The final model achieves precision of 0.97 and recall of 0.92 for human images, and precision of 0.92 and recall of 0.97 for AI images, well surpassing the project's original target of 0.80 for both metrics. The implementation of Grad-CAM visualization further demonstrates that the model has learned meaningful discriminative features rather than spurious correlations.

The project confirms that CNNs trained on large-scale labeled datasets can serve as effective detectors of AI-generated visual content, with practical potential for integration into digital content verification systems.

For future work, the model could be changed to a fine-tuned, pre-trained model (e.g., ResNet50, EfficientNet, or a Vision Transformer) on this dataset would likely push accuracy above 97%, as these architectures have learned rich visual representations from ImageNet. Using data augmentation by applying random flips, rotations, brightness shifts, and JPEG compression artifacts during training would improve robustness to real-world variations. Testing the model on AI images from generators not present in the training data (e.g., newer Stable Diffusion versions) would reveal how well the learned features generalize. Incorporating Frequency Domain Analysis-based features or artifact-specific preprocessing could expose generation artifacts invisible in the spatial domain. Wrapping the trained model in a simple web API or browser extension would demonstrate the practical applicability of this classifier for content moderation.

Works Cited

Sala, A. (n.d.). AI vs. human-generated images [Data set]. Kaggle. https://www.kaggle.com/datasets/alessandrasala79/ai-vs-human-generated-dataset